Виды ошибок. Методы отладки

Принципы тестирования

1. Тестирование демонстрирует наличие дефектов, а не их отсутствие

- Тестирование может показать, что дефекты присутствуют, но не может доказать, что их нет.
- Тестирование снижает вероятность наличия дефектов, находящихся в программном обеспечении, но, даже если дефекты не были обнаружены, тестирование не доказывает его корректности. [ISTQB CTFL]

Например:

Вы тестируете магазин приложений. В рамках тестирования вы нашли 10 дефектов и уверены, что в приложении их больше нет. Приложение успешно отправляется в релиз к конечному пользователю. Конечный пользователь получает сообщение об ошибке и не может зайти в приложение, потому что версия его браузера не поддерживается.

2. Исчерпывающее тестирование недостижимо

Полное тестирование с использованием всех комбинаций вводов и предусловий физически невыполнимо, за исключением тривиальных случаев. Вместо попытки исчерпывающего тестирования должны использоваться анализ рисков, методы тестирования и расстановка приоритетов, чтобы сосредоточить усилия по тестированию. [ISTQB CTFL]

Например:

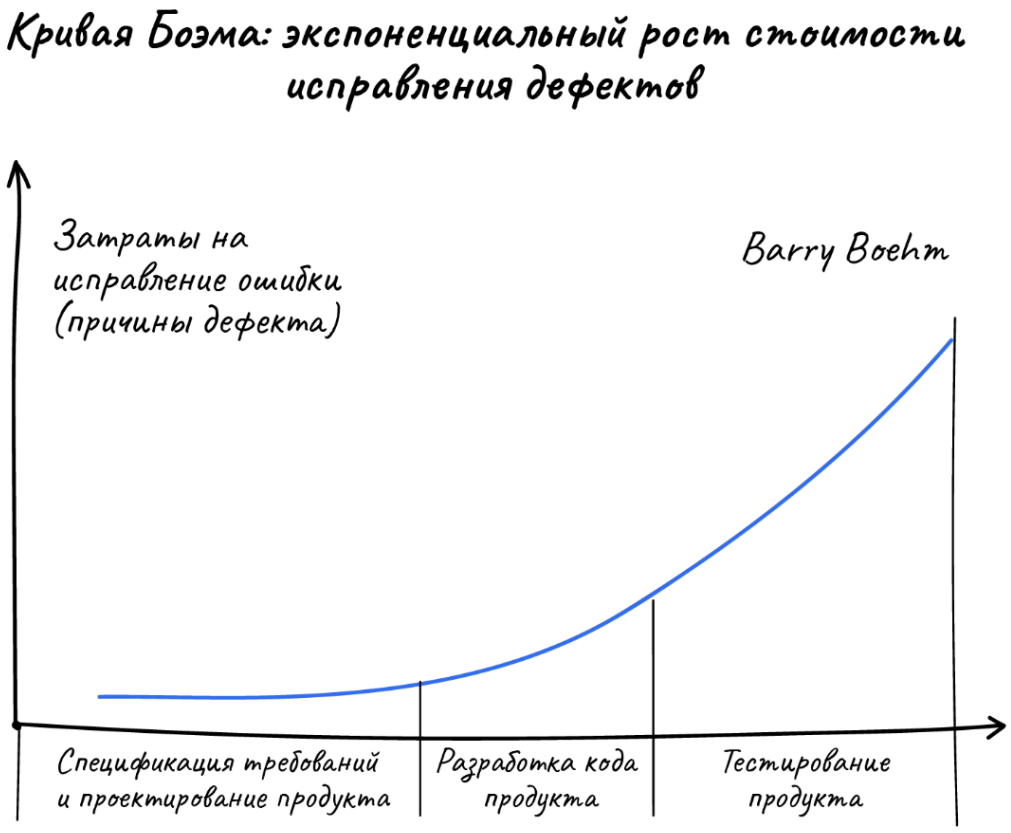
- В вашем приложении используется 15 различных фильтров с возможностью выбора 20 различных параметров в каждом фильтре.
- В итоге вам нужно провести 15*20 = 300 тестов только для фильтрации, а есть еще и другие функции приложения, которые нужно проверить.
- В таком случае тестировщик оптимизирует и приоритизирует свою работу, сокращая объем проверок, например, используя техники тест-дизайна
- Провести исчерпывающее тестирование невозможно.

3. Раннее тестирование сохраняет время и деньги

Для нахождения дефектов на ранних стадиях, как статические, так и динамические активности по тестированию должны быть начаты как можно раньше в жизненном цикле разработки программного обеспечения. Раннее тестирование иногда называют «сдвигом влево».

Тестирование на ранних этапах жизненного цикла разработки программного обеспечения помогает сократить или исключить дорогостоящие изменения. [ISTQB CTFL]

- Существует так называемая кривая Боэма, в которой отражена стоимость исправления дефекта по отношению к фазе разработки конечного продукта. Чем раньше начинается тестирование, тем дешевле исправить дефект. Это и есть графическое отражение принципа раннего тестирования.
- Такой подход называется Shift Left Testing, где тестирование смещается влево относительно всех этапов SDLC и начинается как можно раньше.



4. Кластеризация дефектов

- Обычно небольшое количество модулей содержит большинство дефектов, обнаруженных во время тестирования перед выпуском, или отвечает за большинство эксплуатационных отказов.
- Предсказанные кластеры дефектов и фактические наблюдаемые кластеры дефектов в ходе тестирования или эксплуатации являются важными входными данными для анализа риска, используемого для сосредоточения усилий по тестированию.[ISTQB CTFL]

Пример:

- В таком случае обычно руководствуются частным случаем закона Парето: в 20% модулей приложения сокрыто 80% всех дефектов.
- Например, при тестировании модулей регистрации, авторизации и создания профиля пользователя большая часть дефектов сконцентрирована в модуле авторизации.
- При планировании дальнейшего тестирования мы будем обращать внимание на этот модуль, например, включать его в регрессию (тестирование влияния изменений на другие функции приложения) и учитывать в анализе рисков.
- Другое название этого принципа: скопление дефектов.

5. Парадокс пестицида

- Если одни и те же тесты будут выполняться снова и снова, в конечном счете эти тесты больше не будут находить новых дефектов. Для обнаружения новых дефектов может потребоваться изменение существующих тестов и тестовых данных, а также написание новых тестов. [ISTQB CTFL]

Для борьбы с эффектом пестицида можно использовать следующие **примеры**:

1. Обновление тест-кейсов
2. Обновление тестовых данных
3. Смена ответственного за тестирование определенной функции

6. Тестирование зависит от контекста

- Тестирование выполняется по-разному в зависимости от контекста.
- Например, программное обеспечение управления производством, в котором критически важна безопасность, тестируется иначе, чем мобильное приложение электронной коммерции. [ISTQB CTFL]

7. Заблуждение об отсутствии ошибок

- Некоторые организации ожидают, что тестировщики смогут выполнить все возможные тесты и найти все возможные дефекты, но принципы 2 и 1, соответственно, говорят нам, что это невозможно.
- Тщательное тестирование всех указанных требований и исправление всех обнаруженных дефектов может привести к созданию системы, которая будет трудной в использовании, не будет соответствовать потребностям и ожиданиям пользователей или будет хуже по сравнению с другими конкурирующими системами. [ISTQB CTFL]

- **Например:**
- Компания разработала интернет-магазин для страны, в которой часть населения не может использовать онлайн-оплату. В итоге, конечные пользователи не смогли приобрести желаемый товар, перестали пользоваться приложением и занизили его рейтинг.
- В данном случае при сборе требований компания не учла все кейсы, разработчики создали программу, которая соответствует предоставленным им требованиям, однако не соответствует нуждам пользователя.

Основные понятия и терминология

- **Ошибка (error)** — неверное действие/решение разработчика или пользователя.
- **Дефект / баг (defect, bug, fault)** — дефект в исходном коде/конфигурации, который может привести к отказу.
- **Отказ (failure)** — проявление дефекта в работе системы (сбои, неправильный результат, падение).
- **Инцидент (incident)** — событие, нарушающее нормальную работу сервиса (может быть вызвано дефектом или внешними факторами).

- **Root cause (корневая причина)** — первопричина, давшая начало цепочке событий, ведущих к дефекту/инциденту.
- **Reproducible / Non-reproducible (Heisenbug)** — легко/трудно воспроизводимые ошибки; Heisenbug — «исчезающий» баг, проявляющийся нерегулярно.

Отличие: *ошибка* — причина, *дефект* — код, *отказ* — эффект.

Классификация ошибок

1. По критичности / влиянию

- **Критично (P0 / Sev 1)** — сервис недоступен, данные теряются, безопасность нарушена.
- **Высокая (P1 / Sev 2)** — ключевая функция нерабочая, есть обходной путь.
- **Средняя (P2 / Sev 3)** — некритичная функциональность, косметика.
- **Низкая (P3)** — документация, UX-улучшения.

Важно: **severity** (тяжесть) ≠ **priority** (очередность исправления).

2. По природе и месту возникновения

2.1. Синтаксические ошибки

- Характер: нарушена грамматика языка.
- Проявление: интерпретатор/компилятор не запустит программу или выдаст ошибку парсинга.
- Обнаружение: компиляция/парсинг, статический анализ, IDE.

```
// Пример 1: пропущена точка с запятой
int x = 5
Console.WriteLine(x);

// Пример 2: неверное ключевое слово
publc class MyClass { } // Ошибка: publc → public
```

2.2. Логические ошибки

- Характер: код выполняется без исключений, но результат неверен.
- Проявление: неправильные вычисления, неверные ветвления.
- Обнаружение: тесты (юнит/интеграция), code review, сравнение с эталоном.

```
// Среднее арифметическое (ошибка off-by-one)
int Sum(int[] arr) => arr.Sum();
double Average(int[] arr) => Sum(arr) / (arr.Length - 1.0); // ошибка: -1
```

2.3. Ошибки времени выполнения (runtime errors)

- Характер: ошибки возникающие только при исполнении при определённых входных данных/условиях (деление на ноль, null-pointer, IndexError). Проявляются только во время работы программы при определённых данных.
- Обнаружение: тесты, мониторинг, исключения/stacktrace.

```
int[] numbers = new int[0];
Console.WriteLine(numbers[0]); // System.IndexOutOfRangeException

string s = null;
Console.WriteLine(s.Length);   // System.NullReferenceException

int x = 10, y = 0;
Console.WriteLine(x / y);      // System.DivideByZeroException
```

2.4. Семантические ошибки

- Характер: код корректен как программа, но его поведение не соответствует заданной семантике/требованиям.
- Обнаружение: функциональные тесты, review с участием заказчика.

```
// Требовалось сортировать по возрастанию, но используется OrderByDescending
var sorted = list.OrderByDescending(x => x.Price);
```

2.5. Ошибки проектирования/архитектуры

- Характер: неверная архитектура, жесткие зависимости, трудность расширения/поддержки.
- Проявление: частые регрессии, большие изменения вместо локальных исправлений.
- Обнаружение: ревью архитектуры, долгосрочный анализ техдолга.
- Пример: класс, который напрямую читает/пишет в БД и одновременно обрабатывает UI-логику.

2.6. Интеграционные и конфигурационные ошибки

- Характер: несовместимость версий, неправильная конфигурация окружения, отличия Dev/Prod.
- Обнаружение: интеграционные тесты, контейнеризация, CI, логирование окружения.
- Пример: Неверные настройки соединения с БД, несовместимые версии пакетов NuGet.

```
// appsettings.json
{
  "ConnectionStrings": {
    "Default": "Server=prod-db;Database=App;User=dev;Password=dev;"
  }
}
// На продакшене остаётся dev-строка
```

2.7. Ошибки параллелизма

- Примеры: race conditions (состояния гонки), deadlocks (взаимоблокировки), lost updates (потерянные обновления). Проявляются редко, под нагрузкой.
- Обнаружение: стресс-тесты, статический/динамический анализ (ThreadSanitizer), логика таймингов.

```
class Counter
{
    private int _value = 0;

    public void Increment()
    {
        _value++; // небезопасно в многопоточной среде
    }

    public int Value => _value;
}
```

Исправление:

```
Interlocked.Increment(ref _value);
```

или

```
lock (_lockObj) { _value++; }
```

2.8. Утечки ресурсов (memory leaks, handle leaks)

- Характер: со временем система деградирует, лимит дескрипторов.
- Обнаружение: профайлинг памяти, мониторинг, long-run тесты.
- Пример: Неправильное использование IDisposable.

Неправильно:

```
void ReadFile()
{
    var reader = new StreamReader("data.txt");
    string text = reader.ReadToEnd();
    // reader не закрыт → утечка дескриптора
}
```

Правильно:

```
using (var reader = new StreamReader("data.txt"))
{
    string text = reader.ReadToEnd();
}
```

2.9. Проблемы безопасности (vulnerabilities)

- Примеры: SQL injection, неправильная аутентификация.
- Обнаружение: код-ревью, SAST, DAST, статический анализ на уязвимости, pentest.
- Пример: SQL-инъекции

```
string name = txtName.Text;
string query = $"SELECT * FROM Users WHERE Name = '{name}'";
// Если name = "'; DROP TABLE Users;--"
```

- Решение: параметризованный запрос (SqlParameter, Dapper, EF).

2.10. Производительность и масштабируемость (non-functional)

- Характер: высокая задержка, низкая пропускная способность под нагрузкой.
- Обнаружение: load / stress тесты, профайлеры.
- Пример: Ненужное создание объектов, неэффективные LINQ-запросы, частые обращения к БД.

```
// Пример: многократный вызов ToList() в цикле
foreach (var item in db.Users.ToList()) { ... }
```

2.11. Ошибки данных (data errors)

- Характер: неверные входные данные, неконсистентные записи, проблемы сериализации/десериализации.
- Обнаружение: проверка валидаций, схем (JSON Schema), тесты на граничные / некорректные данные.
- Пример: В JSON возраст строкой, а в модели int → JsonException.

```
string json = @"{""Age"":""abc""}";
try { JsonSerializer.Deserialize<User>(json); }
catch (JsonException e) { Console.WriteLine("Ошибка данных: " + e.Message); }

class User { public int Age { get; set; } }
```

Неконсистентность БД

В таблице заказов поле CustomerId ссылается на несуществующего клиента.

Ошибку можно поймать:
с помощью **внешних ключей** (Foreign Key),

регулярной **валидации данных** при чтении.

Причины появления ошибок

- **Человеческий фактор**: опечатки, неполное понимание задачи, неверные предположения.
- **Неполные/неясные требования**: неясности/двусмысленность в спецификации → неправильная реализация.
- **Сложность системы**: взаимозависимости, распределённость, асинхронность.
- **Инструменты/процессы**: отсутствуют тесты, CI, code review

- **Окружение и конфигурация**: непротестированные комбинации версий, переменные среды.
- **Внешние зависимости**: изменение API, сетевые сбои.
- **Накопление техдолга**: неочищенный код, отсутствует рефакторинг.

Методы обнаружения ошибок

- **Статический анализ / линтеры**: проверка стиля, потенциальных ошибок без запуска. (ESLint, Pylint, SonarQube)

Когда: на ранних этапах разработки, pre-commit, CI.

- **Unit-тестирование**: проверка маленьких частей (функций/классов).

Когда: всегда — минимальный уровень защиты от регрессий.

- **Integration / System / Acceptance тесты**: проверка взаимодействия и работы всей системы.

Когда: при интеграции модулей, перед релизом.

- **Функциональное ручное тестирование / Exploratory testing**: интуитивный поиск багов, удобен для UI/UX.

Когда: UAT (User Acceptance Testing - пользовательское приемочное тестирование) , тестирование новых фич.

- **Load / Stress / Performance testing**: поиск узких мест под нагрузкой.

Когда: при масштабировании, релизе производственных версий.

- **Мониторинг и логирование в продакшене**: быстрый сбор информации об инцидентах.

Когда: для обнаружения реальных проблем пользователей.

- **Fuzzing/Randomized testing**: ввод некорректных/ неожиданных данных. Особенно эффективен для парсеров, протоколов.

Когда: безопасность, парсеры, сетевые протоколы.

- **Distributed tracing**: трассировка запросов через микросервисы (trace-id) для поиска узких мест и ошибок в цепочках вызовов.

Когда: распределённые системы.

- **Обзор кода/ парное программирование**: человеческая проверка, предотвращение логических ошибок.

Когда: всегда рекомендуется.

- **Инструменты для сообщения о сбоях:** сбор stacktrace/события ошибок.

Когда: продакшен.

Отладка (debugging)

Отладка — это контролируемый процесс: от воспроизведения до исправления и предотвращения повторения.

- Ключевая идея: **сначала воспроизвести — затем ломать систему для анализа**. Без воспроизведения правка может скрыть, но не устранить проблему.

Шаги стандартного процесса отладки:

1. **Воспроизведение**: получить минимальный набор шагов/входных данных, при которых баг проявляется.
2. **Сбор контекста**: логи, окружение, версии, конфигурация, стек вызовов.
3. **Локализация**: сузить область кода/модуля, где проявляется ошибка.
4. **Анализ причины**: понять, почему код ведёт себя так (переменные, состояния, потоки).
5. **Разработка исправления**: минимальное, тестируемое изменение.
6. **Проверка**: юнит/интеграционные тесты, регрессия.
7. **Документирование**: баг-репорт, RCA (Root Cause Analysis - анализ первопричин), изменения в тестах/CI.
8. **Предотвращение**: паттерны, тесты, процессы, изменение архитектуры.

Методы и техники отладки

1. Print-debugging (вставка временных выводов)

- **Суть:** вставка print/console.log/printf для отслеживания значений и проходов по коду.
- **Когда:** быстрые ошибки, простые скрипты, когда нет дебаггера под рукой.
- **Плюсы:** максимально простой; работает везде.
- **Минусы:** «мусор» в коде, может изменить тайминг, труден при многопоточности/распределённых системах.
- **Рекомендации:** использовать временные сообщения с контекстом (function name, request id). Удалять/заменять реальным логированием.

```
def compute(a, b):
    print("[DEBUG] compute called: a=%s, b=%s" % (a, b))
    return a / b
```

2. Логирование (structured logging)

- **Суть:** настроенное логирование с уровнями (DEBUG/INFO /WARNING/ERROR/CRITICAL), форматами, идентификаторами запросов.
- **Когда:** приложение в dev/prod, нужно оставить следы работы.
- **Плюсы:** исторические данные, поиск по логам, интеграция с ELK/Graylog.
- **Минусы:** требует дизайна (уровни, объем), хранение логов, обезличивание PII (Personally Identifiable Information).
- **Рекомендации:** всегда логируйте контекст (request_id, user_id), избегайте логирования паролей/PII, используйте ротацию логов.

```
import logging
logger = logging.getLogger(__name__)
logger.info("Start processing", extra={"request_id": req_id})
```

3. Интерактивный отладчик (debugger)

- **Суть:** запуск программы в режиме отладки, точки останова, пошаговое выполнение, просмотр переменных и стека.
- **Когда:** сложные алгоритмы, понимание потока исполнения, нужно изучить состояние в конкретной строке.
- **Инструменты:** pdb, gdb, lldb, IDE (PyCharm, VSCode, IntelliJ), browser devtools.
- **Ключевые команды (gdb):** break <line|func>, run, next (n), step (s), continue (c), bt (backtrace), print <var>.
- **Ключевые команды (pdb):** b <line>, n, s, c, p variable, l.
- **Плюсы:** прямой контроль, удобство локализации.
- **Минусы:** не всегда применим в продакшене, трудно для многопроцессных/распределённых систем.

4. Conditional breakpoints / watchpoints

- **Суть:** останов только при выполнении условия (например, переменная принимает конкретное значение).
- **Когда:** редкие баги, огромные циклы, сложно уловить вручную.
- **Рекомендации:** использовать при отладке гонок/флэйки багов.

5. Post-mortem debugging (core dumps / crash analysis)

- **Суть:** анализ дампа памяти после падения процесса. Используется для C/C++/native приложений, Java heap dumps.
- **Инструменты:** gdb + core dump, jmap/jstack, lldb.
- **Когда:** процесс падает в продакшене/на тесте, нужен детальный анализ состояния.

6. Профилирование (profiling)

- **Суть:** сбор метрик CPU/memory/IO для поиска узких мест.
- **Инструменты:** cProfile (Python), perf, VisualVM (Java), pprof, flamegraphs.
- **Когда:** проблемы производительности, утечки памяти, долгие операции.

7. Sanitizers / Memory checkers

- **Суть:** инструменты для поиска ошибок памяти: use-after-free, buffer overflow, undefined behavior.
- **Инструменты:** Valgrind, AddressSanitizer (ASan), UndefinedBehaviorSanitizer (UBSan), LeakSanitizer.
- **Когда:** C/C++ проекты, подозрения на memory corruption/leak.

8. Фаззинг (fuzzing)

- **Суть:** генерация случайных/структурированных входных данных для обнаружения падений и уязвимостей.
- **Инструменты:** AFL, libFuzzer и т.п.
- **Когда:** парсеры, сериализация, внешние интерфейсы, безопасность.

9. Delta-debugging / git-bisect

- **Суть:** находить коммит, введший регрессию, методично деля пространство коммитов.
- **Команды:** git bisect start, git bisect bad, git bisect good, git bisect run <test-script>.
- **Когда:** регрессия появилась после серии коммитов.

10. Remote debugging / attach

- **Суть:** подключиться к процессу на удаленном хосте/контейнере (attach).
- **Инструменты:** gdb -p PID, jdb, remote IDE attach, python -m pdb for remote via SSH.
- **Риски:** принудительное attach может повлиять на производительность/тайминги; делать осторожно.

11. Replay / deterministic replay

- **Суть:** запись последовательности событий (трасс, запросов) и воспроизведение для отладки.
- **Когда:** распределённые системы, сложно воспроизводимые проблемы.

12 Тесты как отладочный инструмент

- Написание unit-теста/интеграционного теста, который воспроизводит баг — отличный способ локализовать и зафиксировать проблему. Тест служит регресс-тестом после исправления.